

End-to-End AI Governance Classification Pipeline

This document captures the full pseudocode workflow for a hybrid semantic + LLM-based governance classification pipeline, including preprocessing, semantic scoring, threshold routing, governance flagging, storage, and future classifier training.

STEP 0 — Setup (One Time)

```
Load taxonomy JSON

For each domain and intent:
    embed(definition) → store as definition_vector
    for each keyword → embed(keyword) → store as keyword_vectors[]
    for each phrase → embed(phrase) → store as phrase_vectors[]

Load governance_flag_matrix → high_risk[], medium_risk[]
```

STEP 1 — Data Ingestion & Preprocessing

```
Pull logs from source (daily batch or streaming)

For each session_id:
    group all rows by session_id
    sort rows by timestamp / row_id

    for each row in session:
        if len(prompt) < 5 tokens:
            merge with next row in session

    combined_prompt = concatenate all prompts in session

    enrich with metadata:
        department → domain_prior
        emp_type → risk_weight modifier
        country → jurisdiction flag
```

STEP 2 — Hybrid Semantic Scoring

```
For each session (combined_prompt):
    input_vector = embed(combined_prompt)

    For each category in domains[]:
        // Prototype score (50%)
        proto_scores = [cosine(input_vector, p) for p in phrase_vectors]
        prototype_score = max(proto_scores)

        // Keyword score (30%)
        kw_scores = [cosine(input_vector, k) for k in keyword_vectors]
        keyword_score = max(kw_scores)

        // Definition score (20%)
        definition_score = cosine(input_vector, definition_vector)

        domain_score[category] = (
            0.5 * prototype_score +
            0.3 * keyword_score +
            0.2 * definition_score
        )
    // Apply metadata prior
```

```
        if department matches category:
            domain_score[category] *= boost_factor // e.g 1.2

predicted_domain = argmax(domain_score)
domain_confidence = max(domain_score)

// Repeat exact same logic for intents[]
predicted_intent = argmax(intent_score)
intent_confidence = max(intent_score)

combined_confidence = mean(domain_confidence, intent_confidence)
```

STEP 3 — Threshold Decision

```
HIGH_CONFIDENCE_THRESHOLD = 0.75
LOW_CONFIDENCE_THRESHOLD = 0.50

if combined_confidence >= HIGH_CONFIDENCE_THRESHOLD:
    label = (predicted_domain, predicted_intent)
    label_source = "semantic"

elif combined_confidence >= LOW_CONFIDENCE_THRESHOLD:
    route → LLM fallback (STEP 4)

else:
    label = "unclassified"
    route → manual review queue
```

STEP 4 — LLM Fallback

```
For each log routed to LLM:

    truncated_prompt = combined_prompt[:300 characters]

    llm_input = '''
        Classify this AI assistant log into one domain and one intent.

        Domains: {domain list with definitions}
        Intents: {intent list with definitions}

        Log: {truncated_prompt}

        Respond in JSON:
        {
            "domain": "D0X",
            "intent": "I0X",
            "confidence": 0.0-1.0,
            "reason": "one line explanation"
        }
    '''

    response = call_LLM(llm_input)

    label = (response.domain, response.intent)
    label_source = "llm"
    confidence = response.confidence
    reason = response.reason

    // Cost guard
    if total_tokens_used >= BUDGET_CAP:
        stop LLM calls
        remaining logs → manual review queue
```

STEP 5 — Governance Flagging

```
For each labeled log:

    pair = (predicted_domain, predicted_intent)

    if pair in high_risk[]:
        governance_flag = "HIGH"
        reason = high_risk[pair].reason
        route → governance review queue

    elif pair in medium_risk[]:
        governance_flag = "MEDIUM"
        reason = medium_risk[pair].reason
        route → governance monitoring

    else:
        governance_flag = "NONE"

    // Anomaly layer on top
    if any(suspicious_keyword in combined_prompt
        for suspicious_keyword in watchlist):
        governance_flag = escalate to "HIGH"
        flag_reason = "suspicious keyword detected"
```

STEP 6 — Output & Storage

```
For each session write output record:

{
    session_id,
    predicted_domain,
    predicted_intent,
    domain_confidence,
    intent_confidence,
    label_source,
    governance_flag,
    flag_reason,
    department,
    emp_type,
    country,
    timestamp
}

Route to:
governance_flag = HIGH    → governance_alerts table
governance_flag = MEDIUM → monitoring_queue table
label_source = llm        → training_dataset table
all records               → classification_logs table
```

STEP 7 — Classifier Training

```
labeled_data = query training_dataset table

if min(samples_per_category) >= 100:

    features = fused_embed(prompt + response[:200])
               + one_hot(department)
               + one_hot(emp_type)

    train domain_classifier on features → domain_label
    train intent_classifier on features → intent_label

    evaluate on held_out_set (20% stratified split)

    if f1_per_category >= 0.80:
        deploy classifiers
        replace STEP 2 + 3 + 4 with classifier inference
    else:
        collect more labeled data → repeat
```

Pipeline Summary

Pipeline Area	Primary Purpose
Setup	Load embeddings and governance matrices
Preprocessing	Normalize and enrich session logs
Semantic Scoring	Hybrid vector similarity classification
Threshold Routing	Determine semantic vs LLM fallback
LLM Fallback	Handle ambiguous low-confidence cases
Governance	Flag high-risk or suspicious activity
Storage	Persist classifications and queues
Training	Transition to supervised classifiers